

PORTADA

Análisis y Diseño de Sistemas II

Semana 12 — Patrones GoF Creacionales I: Factory Method, Abstract Factory

Universidad de Antioquía · Ingeniería de Sistemas



APERTURA

GRASP les dijo *qué* buscar.
Hoy empieza el catálogo de *cómo* resolverlo, ya inventado.

TRANSICIÓN

De los principios generales a un catálogo concreto

En la **semana 10** vieron los 9 patrones **GRASP** (Larman): guías para decidir a qué clase asignarle cada responsabilidad. En la **semana 11** modelaron ese diseño en UML (diagramas de secuencia/colaboración y de clases de diseño) y aplicaron GRASP en un laboratorio sobre su propio proyecto.

Hoy se abre un catálogo distinto: los **patrones GoF**. Ya no son principios para razonar el diseño desde cero — son **soluciones concretas, con nombre y estructura fija**, para problemas de diseño que se repiten una y otra vez en software orientado a objetos.

CONCEPTO

Qué es un patrón de diseño

Un **patrón de diseño** es una solución reutilizable a un problema que aparece repetidamente en un contexto dado, descrita de forma lo bastante general como para aplicarse en distintos lenguajes y dominios, pero lo bastante concreta como para traducirse directamente en clases, objetos y sus colaboraciones.

No es código que se copia y pega tal cual: es una **estructura de diseño con nombre** (clases, roles, relaciones) que, una vez que un equipo la reconoce, se comunica en una sola palabra — "usemos un Factory Method aquí" — en vez de reexplicar la solución completa cada vez.

CONCEPTO

GoF: el catálogo de Gamma, Helm, Johnson y Vlissides

GoF ("Gang of Four", Pandilla de los Cuatro) es el apodo con el que se conoce a los cuatro autores del libro *Design Patterns: Elements of Reusable Object-Oriented Software* (1994): Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides.

Ese libro cataloga **23 patrones de diseño** documentados con un formato fijo: nombre, problema que resuelve, estructura de clases, código de ejemplo y consecuencias (ventajas y costos de usarlo). Es, hasta hoy, el catálogo de patrones de diseño orientado a objetos más citado y enseñado en la industria.

CONCEPTO · ACLARACIÓN IMPORTANTE

GRASP y GoF no son lo mismo, y no compiten entre sí

	GRASP (Larman)	GoF (Gamma et al.)
Qué es	Principios / guías generales para razonar	Soluciones concretas y catalogadas
Pregunta que responde	"¿A qué clase le asigno esta responsabilidad?"	"¿Qué estructura de clases ya resuelve este problema?"
Resultado	Un criterio de decisión (ej. Information Expert, Creator)	Una plantilla de clases con nombre fijo (ej. Factory Method)
Cuándo se usa	Durante todo el diseño de detalle	Cuando el problema calza con uno ya catalogado

No son alternativos: los patrones GoF a menudo son la **forma concreta** de lograr lo que un principio GRASP pide en abstracto — como van a ver hoy mismo con **Creator**.

CONCEPTO

Los 23 patrones GoF, en tres familias

- **Creacionales:** cómo se crean los objetos, ocultando la decisión de qué clase concreta instanciar. **Se ven hoy y el viernes.**
- **Estructurales:** cómo se componen clases y objetos para formar estructuras más grandes sin duplicar código. Se ven en la **semana 13.**
- **De comportamiento:** cómo colaboran los objetos e intercambian responsabilidades en tiempo de ejecución. Se ven en la **semana 14.**

Hoy, dentro de la familia creacional, se ven los dos primeros: **Factory Method** y **Abstract Factory**.



CONTRA EJEMPLO

El código cliente decidiendo qué clase concreta crear

```
class GeneradorDeReportes:
    def generar(self, tipo, datos):
        if tipo == "pdf":
            reporte = ReportePdf(datos)
        elif tipo == "excel":
            reporte = ReporteExcel(datos)
        elif tipo == "html":
            reporte = ReporteHtml(datos)
        else:
            raise ValueError("tipo no soportado")
        reporte.exportar()
```

`GeneradorDeReportes` conoce y crea directamente cada clase concreta (`ReportePdf` , `ReporteExcel` , `ReporteHtml`) con `new` /instanciación directa dentro de un `if/elif` .

CONCEPTO

Por qué ese `if/elif` es un problema de diseño

Cada vez que aparece un nuevo tipo de reporte (por ejemplo, `"word"`), hay que **modificar** `GeneradorDeReportes` agregando otro `elif` . Eso viola **Open/Closed Principle (OCP)**, visto en la semana 8: la clase debería estar cerrada a modificación y abierta a extensión.

Además, `GeneradorDeReportes` queda **acoplado** a las tres clases concretas de reporte — si el constructor de `ReportePdf` cambia, hay que tocar también el generador. Esto es exactamente el tipo de decisión de "quién crea el objeto" que GRASP llama **Creator** (semana 10): alguien debe crearlo, pero ¿quién y cómo, sin acoplarse a cada clase concreta?

CONCEPTO

Definición: Factory Method

Factory Method define un método —a menudo abstracto, o con una implementación por defecto que las subclases pueden sobrescribir— cuya única responsabilidad es **crear y devolver un objeto**. Cada subclase concreta decide, al sobrescribir ese método, qué clase concreta instanciar.

El código cliente ya no depende de `new ClaseConcreta()` : depende de un **creador** (una clase o interfaz) y de su método de fábrica, sin saber cuál subclase de creador está usando en realidad.

CONCEPTO

Factory Method, en código

```
abstract class GeneradorDeReportes:
    def generar(self, datos):
        reporte = self.crear(datos)    # método de fábrica
        reporte.exportar()

    @abstractmethod
    def crear(self, datos): ...        # las subclases deciden qué crear

class GeneradorPdf(GeneradorDeReportes):
    def crear(self, datos):
        return ReportePdf(datos)

class GeneradorExcel(GeneradorDeReportes):
    def crear(self, datos):
        return ReporteExcel(datos)
```

Agregar un nuevo tipo de reporte ya no exige tocar `GeneradorDeReportes` : se agrega una nueva subclase (`GeneradorWord`) con su propio `crear()` . La clase base queda **cerrada a modificación** y el sistema queda **abierto a extensión**.

CONCEPTO · RELACIÓN CON LO VISTO

Factory Method conecta tres cosas que ya conocen

- **Creator (GRASP, semana 10):** GRASP pregunta en abstracto "¿quién debería crear este objeto?". Factory Method es una **respuesta concreta y catalogada**: una jerarquía de creadores, cada uno responsable de crear su propia familia de productos.
- **OCP (semana 8):** el `if/elif` del contraejemplo obligaba a modificar código existente para extenderlo. Factory Method resuelve eso agregando subclases en vez de editar la clase base.
- **DIP (semana 9):** el código cliente (`generar()`) ya no depende de las clases concretas de reporte — depende del método abstracto `crear()` , una abstracción.

Factory Method no es una idea nueva: es la **forma con nombre y estructura fija** de lograr, a la vez, lo que Creator, OCP y DIP piden por separado.

TRANSICIÓN

¿Y si no es un objeto, sino una familia completa?

Factory Method resuelve la creación de **un** producto a la vez, mediante herencia (una subclase por variante). Pero hay problemas donde no se crea un objeto suelto, sino **varios objetos relacionados que deben ser consistentes entre sí** — por ejemplo, todos los componentes visuales de un mismo tema, o todos los conectores de una misma base de datos.

CONCEPTO

Definición: Abstract Factory

Abstract Factory define una interfaz con **varios métodos de creación**, uno por cada tipo de producto de una familia, sin especificar las clases concretas. Cada implementación concreta de esa interfaz (cada "fábrica concreta") crea **toda una familia de productos que son compatibles entre sí**.

Ejemplo típico: una fábrica de componentes de interfaz gráfica para distintos temas visuales. `FabricaTemaOscuro` crea un `Boton` y una `BarraDeMenu` con el estilo oscuro; `FabricaTemaClaro` crea las mismas dos piezas, pero con el estilo claro. El código cliente pide los componentes a "una fábrica" sin saber cuál, y siempre obtiene una combinación coherente.

CONCEPTO

Abstract Factory, en código

```
interface FabricaDeComponentesUI:
    def crearBoton(self) -> Boton: ...
    def crearBarraDeMenu(self) -> BarraDeMenu: ...

class FabricaTemaOscuro implements FabricaDeComponentesUI:
    def crearBoton(self): return BotonOscuro()
    def crearBarraDeMenu(self): return BarraDeMenuOscura()

class FabricaTemaClaro implements FabricaDeComponentesUI:
    def crearBoton(self): return BotonClaro()
    def crearBarraDeMenu(self): return BarraDeMenuClara()

class Ventana:
    def __init__(self, fabrica: FabricaDeComponentesUI):
        self.boton = fabrica.crearBoton()
        self.menu = fabrica.crearBarraDeMenu()
```

`Ventana` recibe una fábrica (inyectada, igual que en DIP) y arma toda su interfaz llamando a los métodos de esa fábrica — nunca mezcla un `BotonOscuro` con una `BarraDeMenuClara`, porque ambos salen de la misma fábrica concreta.

CONCEPTO · COMPARACIÓN

Factory Method vs. Abstract Factory

	Factory Method	Abstract Factory
Qué crea	Un producto	Varios productos relacionados (una familia)
Mecanismo	Herencia — subclase sobrescribe un método	Composición — un objeto fábrica con varios métodos
Se extiende agregando	Una subclase de creador nueva	Una implementación de fábrica nueva
Garantiza	Que el cliente no dependa de la clase concreta del producto	Que los productos creados juntos sean compatibles entre sí

De hecho, es común que una **Abstract Factory** implemente cada uno de sus métodos de creación **usando** un **Factory Method** internamente — no son excluyentes.

INTERACCIÓN

¿Factory Method o Abstract Factory?

En el chat, para cada caso indiquen cuál de los dos patrones calza mejor:

1. Un sistema de notificaciones que hoy envía solo por correo, pero podría necesitar una subclase que envíe por SMS más adelante — un solo tipo de objeto (el canal).
2. Un sistema que debe generar, coordinados entre sí, un conector de base de datos, un serializador y un validador — distintos para PostgreSQL vs. para MongoDB, pero siempre los tres juntos y compatibles.
3. Una jerarquía de exportadores de reportes (PDF, Excel, HTML) donde cada exportador crea un solo tipo de documento de salida.

5 minutos · respondan en el chat

TENSIÓN CONCEPTUAL

No todo lo que se crea necesita un Factory

Si hoy solo existe **una** clase concreta y el equipo no tiene ninguna razón para prever otra variante en el futuro cercano, crear una jerarquía de creadores o una fábrica abstracta es **sobre-ingeniería**: agrega clases, interfaces e indirección para un problema que no existe todavía.

Esta es la misma tensión que ya vieron con **DIP** (semana 9, "no una interfaz para cada clase") y con **Pure Fabrication** (semana 10, "no crear una clase artificial si no resuelve un problema real"): un patrón de diseño se justifica cuando hay **variación real o prevista**, no por aplicarlo porque "así se ve más profesional". Un simple `new ClaseConcreta()` sigue siendo la opción correcta cuando no hay variantes que anticipar.



SÍNTESIS

Los dos primeros patrones creacionales, en una frase

1. **GoF** es un catálogo de 23 soluciones concretas (Gamma, Helm, Johnson, Vlissides, 1994), organizado en tres familias: creacional, estructural, de comportamiento.
2. **Factory Method**: un método sobrescrito por subclases decide qué clase concreta crear — una jerarquía de creadores, un producto por creador.
3. **Abstract Factory**: una interfaz con varios métodos de creación produce una familia completa de productos compatibles entre sí — composición, no herencia.
4. Ambos son formas concretas de lograr lo que **Creator**, **OCP** y **DIP** ya pedían en abstracto — y ninguno se justifica si no hay variación real que anticipar.



CIERRE

El viernes: los otros tres patrones creacionales

Hoy vieron 2 de los 5 patrones creacionales del catálogo GoF. El **viernes** se completan los otros tres: **Builder** (construcción de objetos complejos paso a paso), **Prototype** (creación de objetos clonando una instancia existente) y **Singleton** (garantizar una única instancia de una clase), junto con un **laboratorio de implementación** aplicando los cinco patrones creacionales al proyecto de curso.

Recuerden: el miércoles de esta semana es Coworking (Review y Retrospectiva del Sprint 2) — no hay sesión teórica nueva ese día.

